2026

C Workshop

Arrays & Strings

Welcome to SESSION 3



Wameedh SC

Agenda

introduction
What are arrays & strings, and why do they matter ?

1- Arrays

1-1 Declaring and initializing
1-2 Accessing elements with indices
1-3 Basic Operations on Arrays (+, -, *, /)
1-4 Searching for value using for loop
1-5 Swapping elements
1-6 Scanning inputs
1-6 Mini exercise

2- Strings

2-1 Null terminator ('\0')
2-2 Safe input (fgets vs scanf)
2-3 Counting characters, vowels, words
2-4 Using <string.h> : strlen, strcpy, strcmp, strcat
2-5 Mini exercise

introduction.

1- Arrays :

An array is a collection of values of the same type, stored next to each other in memory.
Imagine it like a drawer of papers.

```
int main()
{
    int arr[5];

    return 0;
}
```

Why do they matter ? Answer :

- Store lots of data
- Manage it efficiently
- Build meaningful programs

2- Strings :

In C, a string is just a sequence of characters stored in an array of char.
Think of it as a chain of letters placed next to each other in memory.

```c
int main() {
    char word[] = "Hello";

    for (int i = 0; word[i] != '\0'; i++) {
        printf("%c\n", word[i]);
    }

    return 0;
}
```

```
H
e
l
l
o
```

1-1 Declaring and initializing

Structure

```
int numbers[5];                          // type name[size]

int nums[5] = {10, 20, 30, 40, 50};    // You give it the size

int nums[] = {10, 20, 30};             // you let it count for you

int nums[5];                           // index:   0   1   2   3   4
                                       // value:  10  20  30  40  50


nums[0] = 10;
nums[1] = 20;
nums[2] = 30;
nums[3] = 40;
nums[4] = 50;
```

1-2 Accessing elements with indices

Accessing

```c
#include <stdio.h>

int main() {

    int nums[3] = {5, 10, 15};

    printf("First element: %d\n", nums[0]);
    printf("Second element: %d\n", nums[1]);
    printf("Third element: %d\n", nums[2]);

    return 0;
}
```

Changing

```c
#include <stdio.h>

int main() {

    int nums[3] = {5, 10, 15};

    printf("First element: %d\n", nums[0]);
    printf("Second element: %d\n", nums[1]);
    printf("Third element: %d\n", nums[2]);

    nums[0] = 2;
    printf("First element: %d\n", nums[0]);

    return 0;
}
```

1-3 Basic Operations on Arrays (+, -, *, /)

Adding Two Arrays Element-wise

```c
#include <stdio.h>

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int b[5] = {5, 4, 3, 2, 1};
    int sum[5];  // Array to store the result

    for(int i = 0; i < 5; i++) {
        sum[i] = a[i] + b[i];  // Add corresponding elements
    }

    printf("Sum of arrays: ");
    for(int i = 0; i < 5; i++) {
        printf("%d ", sum[i]);
    }

    return 0;
}
```

Adding a number to every element

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    // Add 10 to every element
    for(int i = 0; i < 5; i++) {
        arr[i] = arr[i] + 10;
    }

    printf("Updated array: ");
    for(int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

1-3 Basic Operations on Arrays (+, -, *, /)

Sum of All Elements

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int sum = 0;

    for(int i = 0; i < 5; i++) {
        sum += arr[i];  // sum = sum + arr[i]
    }

    printf("Sum of all elements: %d\n", sum);
    return 0;
}
```

Product of All Elements

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int product = 1;

    for(int i = 0; i < 5; i++) {
        product *= arr[i];
    }

    printf("Product of all elements: %d\n", product);
    return 0;
}
```

1-4 Searching for value using for loop

```c
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int target = 30;  // value to search
    int found = 0;    // flag to indicate if value is found

    for(int i = 0; i < 5; i++) {
        if(arr[i] == target) {
            printf("Value %d found at index %d\n", target, i);
            found = 1;
            break; // exit loop after finding the value
        }
    }
    return 0;
}
```

1-5 Swapping elements

1-5-1 Swapping 2 elements :

The rule

```
temp = arr[index1];
arr[index1] = arr[index2];
arr[index2] = temp;
```

Example

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int temp;
    printf("Original array: ");
    for(int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    // Swap first and last elements
    temp = arr[0];
    arr[0] = arr[4];
    arr[4] = temp;
    printf("Array after swapping first and last: ");
    for(int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

1-5 Swapping elements

1-5-1 Swapping all elements :

```c
int main() {
    int arr[6] = {1, 2, 3, 4, 5, 6};
    int n = 6;  // size of the array
    int temp;
    printf("Original array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    // Swap elements from both ends
    for(int i = 0; i < n/2; i++) {
        temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }
    printf("Reversed array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

1-6 Scanning inputs

```c
#include <stdio.h>
int main() {
    int arr[5];
    printf("Enter 5 integers: ");
    for(int i = 0; i < 5; i++) {
        scanf("%d", &arr[i]); // & is required for scanf
    }
    printf("You entered: ");
    for(int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

1-7 Mini exercise

Design a program in which you ask the user :

1- How many numbers does he want to enter

2- Read these values

3- Swape the first & last elements

4- Print the new array

2- Strings :

2-1 Null terminator ('\0')

```c
#include <stdio.h>
int main() {
    // WRONG: Missing the null terminator '/0'
    char badWord[3] = {'C', 'A', 'T'};
    printf("Without null terminator: %s\n", badWord);
    //  CORRECT: With null terminator
    char goodWord[4] = {'C', 'A', 'T', '\0'};
    printf("With null terminator: %s\n", goodWord);
    //  EASIER & SAFEST: Let C add '\0' automatically
    char autoWord[] = "CAT";
    printf("Automatic null terminator: %s\n", autoWord);
    return 0;
}
```

2-2 Safe input (fgets vs scanf)

fgets Notes :

- fgets() reads a whole line, including spaces.
- It is safer than scanf because it respects the buffer size.
- Syntax: fgets(array, sizeof(array), stdin);
- Adds the null terminator \0 automatically.
- Keeps the newline \n if there is space.
- Use after scanf only if you clear the leftover newline first.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char name[30];
    char sentence[60];

    printf("Enter your first name (no spaces): ");
    scanf("%s", name);
    printf("You entered (scanf): %s\n", name);

    // Clear leftover newline from scanf
    int c;
    while ((c = getchar()) != '\n' ) {}

    printf("Enter a sentence: ");
    fgets(sentence, sizeof(sentence), stdin);

    printf("You entered (fgets): %s", sentence);

    return 0;
}
```

2-3 Counting characters, vowels & words

```c
int main() {
    char text[100];
    int chars = 0;
    int vowels = 0;
    int words = 0;

    printf("Enter a sentence: ");
    fgets(text, sizeof(text), stdin);

    // Loop through the string
    for (int i = 0; text[i] != '\0'; i++) {

        // --- Count characters (ignore newline) ---
        if (text[i] != '\n')
            chars++;

        // --- Count vowels ---
        if (text[i]=='a'||text[i]=='e'||text[i]=='i'||text[i]=='o'||text[i]=='u'||
            text[i]=='A'||text[i]=='E'||text[i]=='I'||text[i]=='O'||text[i]=='U')
            vowels++;
```

2-3 Counting characters, vowels & words

```c

        // --- Count words (spaces + 1) ---
        if (text[i] == ' ')
            words++;
    }

    // If the sentence is not empty
    if (text[0] != '\n' && text[0] != '\0')
        words++;

    // --- Print results ---
    printf("\nCharacters: %d\n", chars);
    printf("Vowels: %d\n", vowels);
    printf("Words: %d\n", words);
    return 0;
}
```

2-4 Using <string.h> : strlen, strcpy, strcmp, strcat

What is #include <string.h> ?

- It is a header file in C.

```
#include <stdio.h>
#include <string.h>
```

- It gives your program access to string functions such as:

  strlen() → count characters
  strcpy() → copy strings
  strcmp() → compare strings
  strcat() → join strings

- Without including <string.h>, these functions do not exist for your program.

2-4 Using <string.h> : strlen, strcpy, strcmp, strcat

```c
#include <stdio.h>
#include <string.h>
int main() {
    char s1[50] = "Hello";
    char s2[50] = "World";
    char copy[50];
    // strlen
    printf("Length of s1: %d\n", (int)strlen(s1));
    // strcpy
    strcpy(copy, s1);
    printf("Copy of s1: %s\n", copy);
    // strcmp
    if (strcmp(s1, s2) == 0)
        printf("s1 and s2 are equal\n");
    else
        printf("s1 and s2 are different\n");
    // strcat
    strcat(s1, s2);
    printf("Concatenation: %s\n", s1);
    return 0;
}
```

2-5 Mini exercise

Write a program that:

1. Asks the user to enter two words.
2. Prints the length of each word (strlen).
3. Looks for vowels in it.

Homework

Write a C program that:

1. Asks the user how many students (max 20).
2. For each student, read:
   - their name (string, with spaces using ‘fgets’)
   - their age (integer)
3. Store:
   - names in an array of strings
   - ages in an array of integers
4. After entering all data, your program must:
   - print the oldest student
   - print the youngest student
   - print the longest name (use strlen)
   - print all students whose name starts A letter.

Send your codes to my email : yanisboub06@gmail.com

Thank you for your attention

Powered by Wameedh SC



Yanis Boubekeur